OWASP juiceshop

Opgavebeskrivelse

I de næste to uger skal I stifte bekendtskab med termerne *vulnerabilities, exploitation*, og *hardening* inden for IT-sikkerhed. Termerne er centrale, når man skal tænke kritisk over, hvordan software, og de tilhørende organisatoriske, strukturelle og menneskelige interaktioner, kan udnyttes af udefrakommende personer eller digitale virusser i vores teknologiske hverdag

Til det formål, skal I arbejde med en kodebase udviklet af organisationen OWASP (Open Worldwide Application Security Project), der deler information og ressourcer til at styrke IoT, system software og webapplikations-sikkerhed

Formålet med opgaven er at I får praktisk erfaring med:

- Identifikation af sårbarheder i en kodebase og webapplikation.
- Udnyttelse af sårbarheder for at forstå konsekvenserne.
- Udarbejdelse af løsningsforslag og anbefalinger til at hærde applikationen.

OWASP har udviklet en fiktiv Juice-Shop, der er en intentionelt usikker webapplikation, som indeholder sårbarheder fra hele OWASP's Top-10, samt andre fejl, der findes i virkelige applikationer, og som I potentielt også selv har eller får i jeres software projekter!

Gode ting at installere:

- Node.js og npm installeret, eller Docker installeret.

Opsætning af Juice Shop:

Option A — Kør fra kilden med Node.js (tættere på kodebasen)

1. Installer Node.js (LTS).
2. Klon projektet:
3. git clone https://github.com/juice-shop/juice-shop.git --depth 1

4. cd juice-shop
5. npm install
6. npm start
7. Åbn http://localhost:3000 i din browser.

Option B — Kør via Docker (mere visuel)

1. Installer Docker.
2. Kør:
3. docker pull bkimminich/juice-shop
4. docker run --rm -p 127.0.0.1:3000:3000 bkimminich/juice-shop
5. Åbn http://localhost:3000 i din browser.

Afleveringsformat

I skal forberede en 10-15 minutters præsentation af jeres arbejde. Her kan I eksempelvis inkludere:

- Oversigt over de fundne sårbarheder (navn, OWASP-kategori, vurderet risiko).
- Trinvis udnyttelsesbeskrivelse for hver sårbarhed inkl. Beviser.
- Konkrete løsningsforslag og prioriteret plan for udbedring.
- Kort refleksion over hvordan sikker udviklingspraksis bør implementeres i organisationen (fx secure-by-design, code review, sikkerhedstests i pipeline).

Opgaver

Opgave 1: Find sårbarheder

Identificér mindst tre konkrete sårbarheder i applikationen. For hver sårbarhed skal du:

1. Beskrive, hvordan du fandt sårbarheden (metode, værktøjer, trin).

2. Demonstrere en udnyttelse (proof-of-concept) - fx via HTTP-forespørgsler, inputeksempler eller skærmbilleder
3. Dokumentere bevis (screenshot, logudsnit, eller resultat i Juice-Shop fx. badge/point)

Opgave 2: Udnyt sårbarheder

Forsøg at udnytte sårbarheder, brug nedenstående eksempler som inspiration:

- Cross-Site Scripting (XSS) — fx indsæt brugerinput, der fremkalder JavaScript.
- SQL Injection — fx loginforsøg med SQL-payloads.
- Broken Authentication — fx adgang til andre brugeres data eller administrative funktioner.
- Sensitive Data Exposure — fx data i klar tekst eller konfigurationsfiler.
- Security Misconfiguration — fx åbne endpoints uden autentifikation.

Bemærk: Juice Shop viser ofte “challenges” eller badges, når en sårbarhed udnyttes korrekt; disse kan bruges som dokumentation!

Opgave 3: Analyse af afbødende foranstaltninger

For hver fundet sårbarhed skal du:

1. Forklare årsagen (hvor i koden eller konfigurationen sårbarheden ligger).
2. Foreslå konkrete afbødende tiltag, fx:
    - Inputvalidering og output encoding (context-aware escaping).
    - Parameteriserede forespørgsler i stedet for string-sammensatte SQL-queries.
    - Streng adgangskontrol og autorisation på API-endpoints.
    - Begrænsning og håndtering af fejlmeddelelser (ingen sensitive oplysninger i fejl).
    - Implementering af sikkerhedsheaders (CSP, HSTS, X-Content-Type-Options, X-Frame-Options).
    - Kryptering af følsomme data i hvile og under transport (TLS).

Opgave 4 (valgfri): Ekstra udfordringer

- Implementér én af dine foreslåede patches direkte i Juice Shop-kodebasen og demonstrér, at sårbarheden er lukket.
- Tilføj et simpelt sikkerhedstjek i npm-scripts (fx npm audit) og dokumentér resultaterne.
- Opret en test i Cypress eller lignende, der verificerer, at en bestemt sårbarhed ikke længere er til stede.

Pensum og Ressourcer

- OWASP Juice Shop — https://owasp-juice.shop
- Pwning OWASP Juice Shop (companion guide) — https://pwning.owasp-juice.shop/
- OWASP Top 10 — https://owasp.org/www-project-top-ten/
- Node.js Security Best Practices — https://nodejs.org/en/security

---

God arbejdslyst!